11. Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int a[], int low, int mid, int high)
{
   int i = low, j = mid + 1, k = 0;

   int* temp = (int*)malloc((high - low + 1) * sizeof(int));

   while (i <= mid && j <= high)
   {
      if (a[i] < a[j])
         temp[k++] = a[i++];
      else
         temp[k++] = a[j++];
   }

   while (i <= mid)
      temp[k++] = a[i++];

   while (j <= high)
      temp[k++] = a[j++];

   // Copy back to original array
   for (i = low, k = 0; i <= high; i++, k++)
      a[i] = temp[k];

   free(temp);
}

void mergesort(int a[], int low, int high)
{
   if (low < high)
   {
      int mid = (low + high) / 2;

      mergesort(a, low, mid);
      mergesort(a, mid + 1, high);
      merge(a, low, mid, high);
   }
}

int main()
{
   int n, i;
   int* a;

   clock_t start, end;
   double time_taken;

   printf("Enter number of elements: ");
   scanf("%d", &n);
```

```
    // Dynamic memory allocation
    a = (int*)malloc(n * sizeof(int));

    srand(time(0));

    // Generate random numbers
    for (i = 0; i < n; i++)
        a[i] = rand() % 100;

    printf("Generated elements:\n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    // Start time
    start = clock();

    mergesort(a, 0, n - 1);

    // End time
    end = clock();

    printf("\nSorted elements:\n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    // Time calculation
    time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nTime taken = %f seconds\n", time_taken);

    free(a);

    return 0;
}
```